```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import StandardScaler

# Q1: Introduction to AI/ML/DL




# Loading the Dataset
print("Loading the Iris dataset...")
iris = load_iris()
X, y = iris.data, iris.target
```

```
Loading the Iris dataset...
```

```python
# Preping the Data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Splitting data.
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42, stratify=y)
print(f"Data split: {X_train.shape[0]} training, {X_test.shape[0]}
testing.")
```

```
Data split: 120 training, 30 testing.
```

```python
# Building and Testing the Model

# Dictionary to hold the results.
results = {}

# a) Logistic Regression
print("\nTraining Logistic Regression...")
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)
y_pred_lr = log_reg.predict(X_test)
results['Logistic Regression'] = {'acc': accuracy_score(y_test,
y_pred_lr),
                                  'cm': confusion_matrix(y_test,
y_pred_lr)}
```

```
Training Logistic Regression...
```

```python
# Decision Tree
print("Training Decision Tree...")
dec_tree = DecisionTreeClassifier(random_state=42)
dec_tree.fit(X_train, y_train)
y_pred_dt = dec_tree.predict(X_test)
results['Decision Tree'] = {'acc': accuracy_score(y_test, y_pred_dt),
                            'cm': confusion_matrix(y_test,
y_pred_dt)}
```

```
Training Decision Tree...
```

```python
# Simple Neural Net
print("Training Neural Network...")
mlp_nn = MLPClassifier(hidden_layer_sizes=(10,), max_iter=1000,
random_state=42)
mlp_nn.fit(X_train, y_train)
y_pred_nn = mlp_nn.predict(X_test)
results['Neural Network'] = {'acc': accuracy_score(y_test, y_pred_nn),
                             'cm': confusion_matrix(y_test,
y_pred_nn)}
print("All models trained!")
```

```
Training Neural Network...
All models trained!
```

```python
print("\n--- Results Summary ---")

for name, res in results.items():
    print(f"\nClassifier: {name}")
    print(f"  Accuracy: {res['acc']:.4f}")
    print(f"  Confusion Matrix:\n{res['cm']}")
```

```
--- Results Summary ---

Classifier: Logistic Regression
  Accuracy: 0.9333
  Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classifier: Decision Tree
  Accuracy: 0.9333
  Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classifier: Neural Network
```

```
  Accuracy: 0.9667
  Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]
```

```
# What makes the Neural Net different?

# 1) The Neural Network models more complex patterns using its hidden
layer, which allows it to generalize well on unseen data.
# 2) Unlike Logistic Regression, which is linear, and Decision Trees,
which make hard splits based on rules, the neural network uses
continuous, non-linear activation functions.
# 3) This makes the neural net better at capturing non-linear
relationships in the data, especially when there’s more complexity or
noise.
# 4) However, neural nets require more training time, tuning, and data
scaling, and are often less interpretable than trees.

# Q2: Mathematics for Deep Learning



# creating two 3x3 matrices
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

B = np.array([[9, 8, 7],
              [6, 5, 4],
              [3, 2, 1]])

print("Matrix A:\n", A)
print("Matrix B:\n", B)
```

```
Matrix A:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]
Matrix B:
 [[9 8 7]
 [6 5 4]
 [3 2 1]]
```

```
# Matrix multiplication (A * B)
mat_mul_result = np.dot(A, B)
# Or we could use A @ B
print("\nMatrix Multiplication (A * B):\n", mat_mul_result)
```

```
Matrix Multiplication (A * B):
```

```
 [[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]
```

```python
# I am using a different matrix for finding determinant/inverse,
because det(A) is 0!
C = np.array([[2, 1, 0],
              [1, 3, 1],
              [0, 1, 1]])
print("\nMatrix C:\n", C)
```

```
Matrix C:
 [[2 1 0]
 [1 3 1]
 [0 1 1]]
```

```python
# Determinant
det_C = np.linalg.det(C)
print(f"\nDeterminant of C: {det_C:.2f}")
```

```
Determinant of C: 3.00
```

```python
# Inverse (if exists)
if det_C != 0:
    inv_C = np.linalg.inv(C)
    print("\nInverse of C:\n", inv_C)
else:
    print("\nInverse of C does not exist (Determinant is zero).")
```

```
Inverse of C:
 [[ 0.66666667 -0.33333333  0.33333333]
 [-0.33333333  0.66666667 -0.66666667]
 [ 0.33333333 -0.66666667  1.66666667]]
```

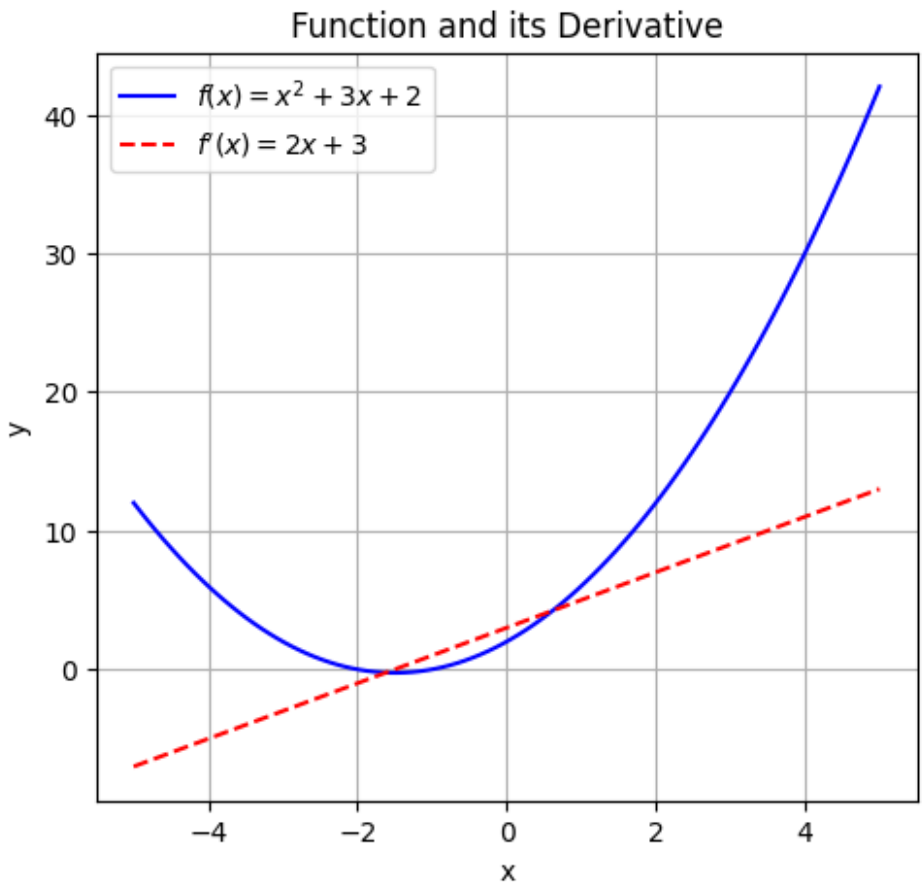
```python
# Eigenvalues and eigenvectors (using matrix C)
eigenvalues, eigenvectors = np.linalg.eig(C)
print("\nEigenvalues of C:\n", eigenvalues)
print("\nEigenvectors of C:\n", eigenvectors)
```

```
Eigenvalues of C:
 [3.87938524 1.65270364 0.46791111]

Eigenvectors of C:
 [[-0.44909879  0.84402963  0.29312841]
 [-0.84402963 -0.29312841 -0.44909879]
 [-0.29312841 -0.44909879  0.84402963]]
```

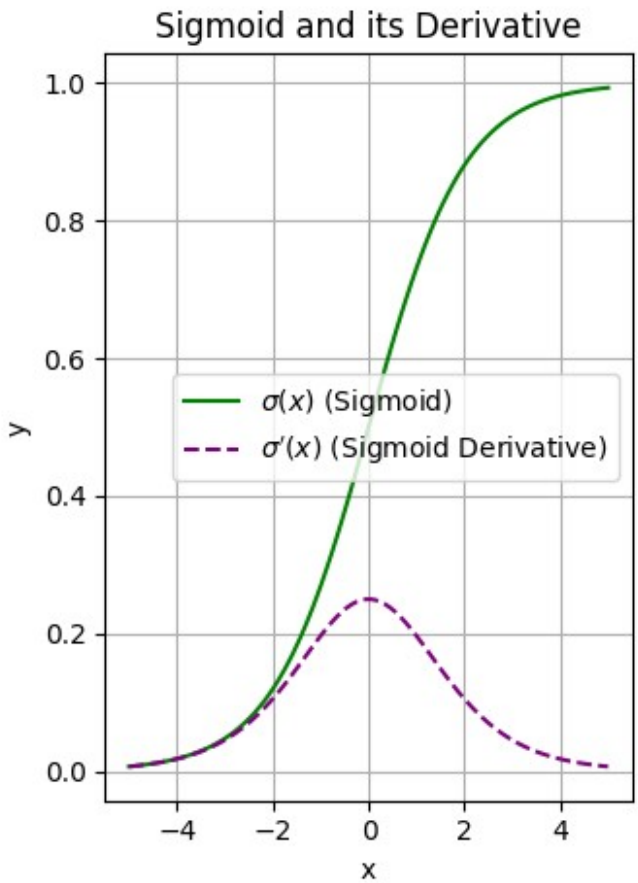
```
# Plotting Functions and Derivatives

print("\nPlotting ")

# Define our range for x
x_vals = np.linspace(-5, 5, 100)

def f(x):
    return x**2 + 3*x + 2

def f_derivative(x):
    return 2*x + 3

y_f = f(x_vals)
y_f_prime = f_derivative(x_vals)
```

Plotting

```
# b) Sigmoid and its derivative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)

y_sig = sigmoid(x_vals)
y_sig_prime = sigmoid_derivative(x_vals)

plt.figure(figsize=(12, 5))

# Plot 1: f(x) and its derivative
plt.subplot(1, 2, 1) # 1 row, 2 columns, 1st plot
plt.plot(x_vals, y_f, label='$f(x) = x^2 + 3x + 2$', color='blue')
plt.plot(x_vals, y_f_prime, label="$f'(x) = 2x + 3$", color='red',
linestyle='--')
plt.title('Function and its Derivative')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
```




```
# Plot 2: Sigmoid and its derivative
plt.subplot(1, 2, 2) # 1 row, 2 columns, 2nd plot
plt.plot(x_vals, y_sig, label='$\sigma(x)$ (Sigmoid)', color='green')
plt.plot(x_vals, y_sig_prime, label="$\sigma'(x)$ (Sigmoid
Derivative)", color='purple', linestyle='--')
plt.title('Sigmoid and its Derivative')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```




```
# Define the distributions P and Q
P = np.array([0.2, 0.5, 0.3])
Q = np.array([1/3, 1/3, 1/3]) # Uniform distribution

print(f"Distribution P: {P}")
print(f"Distribution Q: {Q}")
```

```
Distribution P: [0.2 0.5 0.3]
Distribution Q: [0.33333333 0.33333333 0.33333333]
```

```
cross_entropy = -np.sum(P * np.log(Q))
print(f"\nCross-Entropy H(P, Q): {cross_entropy:.4f}")
```

```
Cross-Entropy H(P, Q): 1.0986
```

```
kl_divergence = np.sum(P * (np.log(P) - np.log(Q)))
print(f"KL-Divergence D_KL(P || Q): {kl_divergence:.4f}")
```

```
KL-Divergence D_KL(P || Q): 0.0690
```

```
# Q3: Neural Networks Basics
```

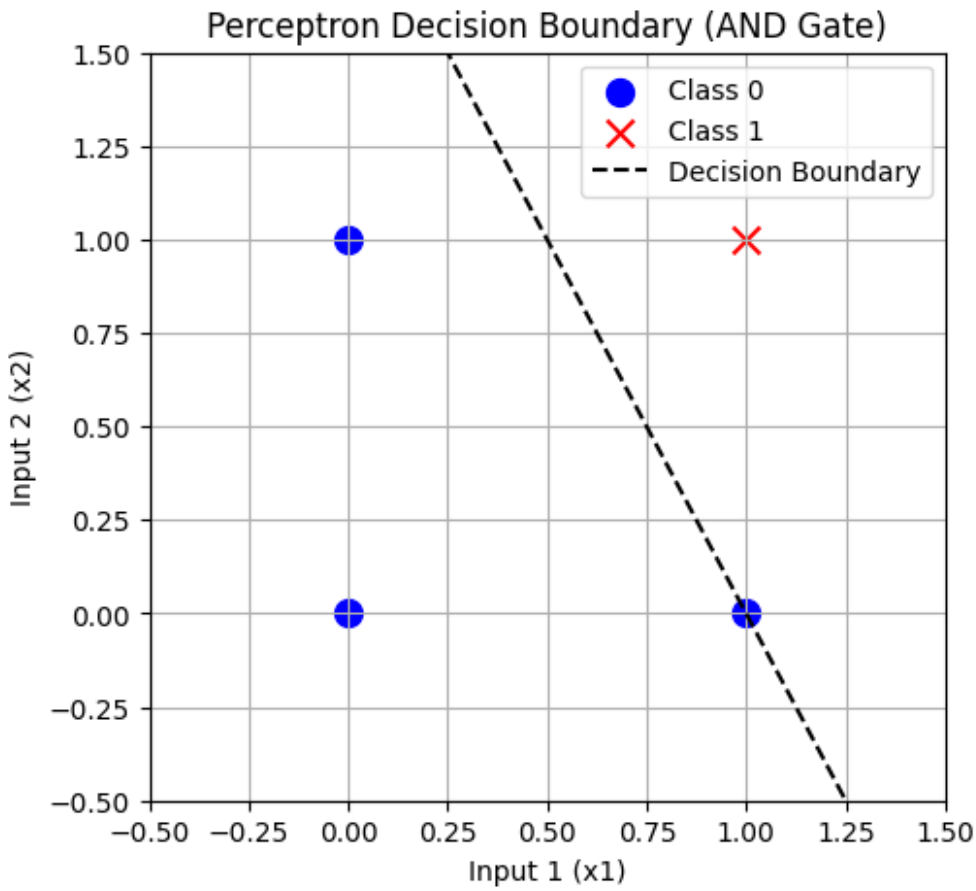
```python
#Implementing a custom perceptron

class Perceptron:
    def __init__(self, learning_rate=0.1, n_epochs=100):
        self.lr = learning_rate
        self.epochs = n_epochs
        self.weights = None
        self.bias = None
        self.errors_ = []

    def _step_function(self, z):
        return np.where(z >= 0, 1, 0)

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0.0

        print("Starting Perceptron training...")
        for epoch in range(self.epochs):
            epoch_errors = 0
            for i in range(n_samples):
                linear_output = np.dot(X[i], self.weights) + self.bias
                y_predicted = self._step_function(linear_output)
                update = self.lr * (y[i] - y_predicted)
                self.weights += update * X[i]
                self.bias += update
                epoch_errors += int(update != 0.0)

            self.errors_.append(epoch_errors)
            if epoch_errors == 0:
                print(f"Converged after {epoch+1} epochs.")
                return
        print(f"Training finished after {self.epochs} epochs (may not
have converged).")

    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias
        return self._step_function(linear_output)

    def net_input(self, X):
        return np.dot(X, self.weights) + self.bias

# Training it on a linearly separable binary dataset (AND gate)
X_and = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_and = np.array([0, 0, 0, 1])

perceptron_and = Perceptron(learning_rate=0.1, n_epochs=100)
```

```
perceptron_and.fit(X_and, y_and)

print(f"\nTrained Weights for AND: {perceptron_and.weights}")
print(f"Trained Bias for AND: {perceptron_and.bias}")
print(f"Predictions for AND gate inputs {X_and.tolist()}:
{perceptron_and.predict(X_and)}")
```

```
Starting Perceptron training...
Converged after 4 epochs.

Trained Weights for AND: [0.2 0.1]
Trained Bias for AND: -0.20000000000000004
Predictions for AND gate inputs [[0, 0], [0, 1], [1, 0], [1, 1]]: [0 0
0 1]
```

```
print("\nVisualizing input points and decision boundary")
plt.figure(figsize=(18, 5))
ax1 = plt.subplot(1, 3, 1)
ax1.scatter(X_and[y_and == 0, 0], X_and[y_and == 0, 1], color='blue',
marker='o', s=100, label='Class 0')
ax1.scatter(X_and[y_and == 1, 0], X_and[y_and == 1, 1], color='red',
marker='x', s=100, label='Class 1')

w = perceptron_and.weights
b = perceptron_and.bias
x1_min, x1_max = -0.5, 1.5
x1_line = np.array([x1_min, x1_max])

if w[1] != 0:
    x2_line = (-w[0] * x1_line - b) / w[1]
    ax1.plot(x1_line, x2_line, 'k--', label='Decision Boundary')
elif w[0] != 0:
    x1_val_for_vertical_line = -b / w[0]
    ax1.axvline(x=x1_val_for_vertical_line, color='k', linestyle='--',
label='Decision Boundary')
else:
    print("Warning: Both weights are zero, cannot plot boundary line
effectively.")

ax1.set_title('Perceptron Decision Boundary (AND Gate)')
ax1.set_xlabel('Input 1 (x1)')
ax1.set_ylabel('Input 2 (x2)')
ax1.set_xlim(x1_min, x1_max)
ax1.set_ylim(x1_min, x1_max)
ax1.legend()
ax1.grid(True)
```

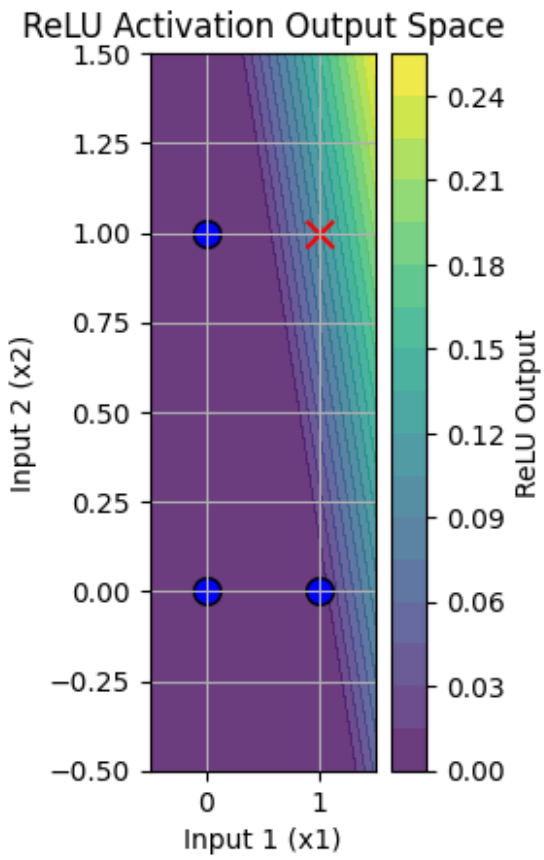
```
Visualizing input points and decision boundary
```




```
# Adding ReLU and Sigmoid activations, and show how they change the
output space.
print("Visualizing how ReLU and Sigmoid change the output space")
def relu_activation(z):
    return np.maximum(0, z)

def sigmoid_activation(z):
    return 1 / (1 + np.exp(-z))

xx1, xx2 = np.meshgrid(np.linspace(x1_min, x1_max, 100),
                       np.linspace(x1_min, x1_max, 100))
grid_points = np.c_[xx1.ravel(), xx2.ravel()]
Z_linear = perceptron_and.net_input(grid_points)
Z_relu = relu_activation(Z_linear).reshape(xx1.shape)
Z_sigmoid = sigmoid_activation(Z_linear).reshape(xx1.shape)

# Plot 2: ReLU Output Space
ax2 = plt.subplot(1, 3, 2)
contour_relu = ax2.contourf(xx1, xx2, Z_relu, levels=20,
cmap='viridis', alpha=0.8)
plt.colorbar(contour_relu, ax=ax2, label='ReLU Output')
ax2.scatter(X_and[y_and == 0, 0], X_and[y_and == 0, 1], color='blue',
```

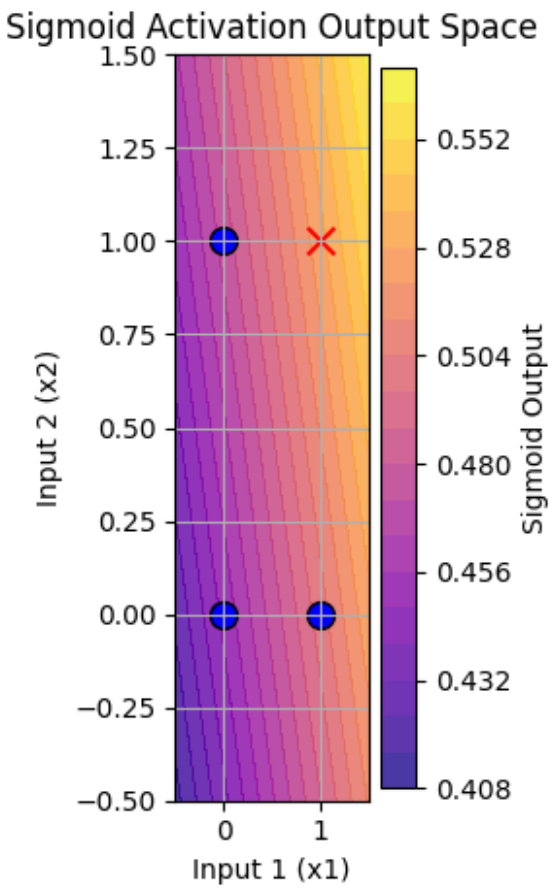
```
marker='o', s=100, edgecolor='k')
# *** FIX: Removed edgecolor='k' from the line below ***
ax2.scatter(X_and[y_and == 1, 0], X_and[y_and == 1, 1], color='red',
marker='x', s=100)
ax2.set_title('ReLU Activation Output Space')
ax2.set_xlabel('Input 1 (x1)')
ax2.set_ylabel('Input 2 (x2)')
ax2.grid(True)
```

Visualizing how ReLU and Sigmoid change the output space




```
# Plot 3: Sigmoid Output Space
ax3 = plt.subplot(1, 3, 3)
contour_sigmoid = ax3.contourf(xx1, xx2, Z_sigmoid, levels=20,
cmap='plasma', alpha=0.8)
plt.colorbar(contour_sigmoid, ax=ax3, label='Sigmoid Output')
ax3.scatter(X_and[y_and == 0, 0], X_and[y_and == 0, 1], color='blue',
marker='o', s=100, edgecolor='k')
ax3.scatter(X_and[y_and == 1, 0], X_and[y_and == 1, 1], color='red',
marker='x', s=100)
ax3.set_title('Sigmoid Activation Output Space')
ax3.set_xlabel('Input 1 (x1)')
```

```
ax3.set_ylabel('Input 2 (x2)')
ax3.grid(True)

plt.tight_layout()
plt.show()
```



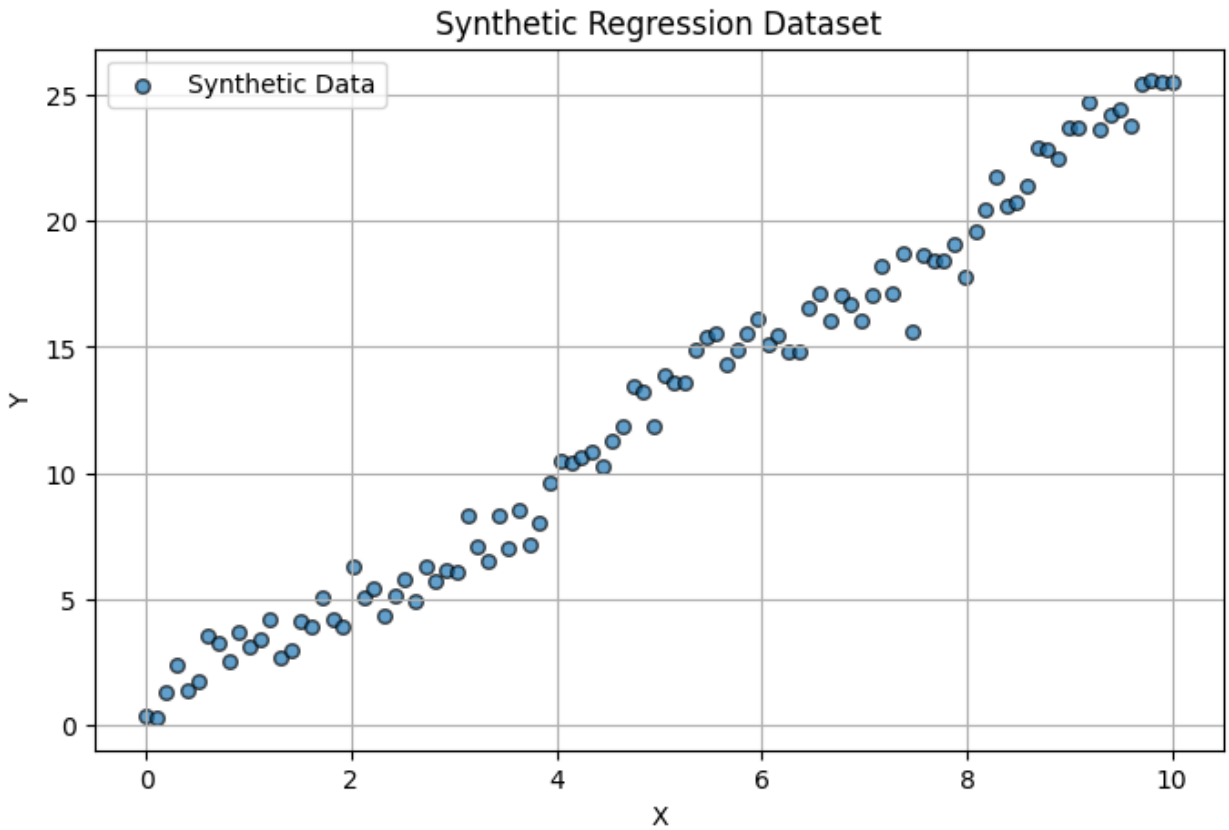

```
print("\nOutput values for AND gate inputs using different activations
on perceptron's net input:")
print("Input | Net Input | Step | ReLU  | Sigmoid")
print("------|-----------|------|-------|---------")
for i in range(len(X_and)):
    net_in = perceptron_and.net_input(X_and[i].reshape(1,-1))[0]
    step_out = perceptron_and._step_function(net_in)
    relu_out = relu_activation(net_in)
    sig_out = sigmoid_activation(net_in)
    print(f"{X_and[i]} | {net_in:9.2f} | {step_out:4.0f} |
{relu_out:5.2f} | {sig_out:7.4f}")
```

```
Output values for AND gate inputs using different activations on
perceptron's net input:
Input | Net Input | Step | ReLU  | Sigmoid
```

```
------|-----------|------|-------|---------
[0 0] |     -0.20 |    0 |  0.00 |  0.4502
[0 1] |     -0.10 |    0 |  0.00 |  0.4750
[1 0] |     -0.00 |    0 |  0.00 |  0.5000
[1 1] |      0.10 |    1 |  0.10 |  0.5250
```

```python
# Q4: Training Neural Networks

#  Creating a synthetic regression dataset
print("Generating synthetic dataset")
np.random.seed(42) # for reproducibility
X = np.linspace(0, 10, 100).reshape(-1, 1)
noise = np.random.randn(100, 1) * 0.8
Y = 2.5 * X + np.sin(X * 1.5) + noise


# Plotting the dataset just for visualisation
plt.figure(figsize=(8, 5))
plt.scatter(X, Y, label='Synthetic Data', alpha=0.7, s=30,
edgecolors='k')
plt.title('Synthetic Regression Dataset')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid(True)
plt.show()
```

```
Generating synthetic dataset
```



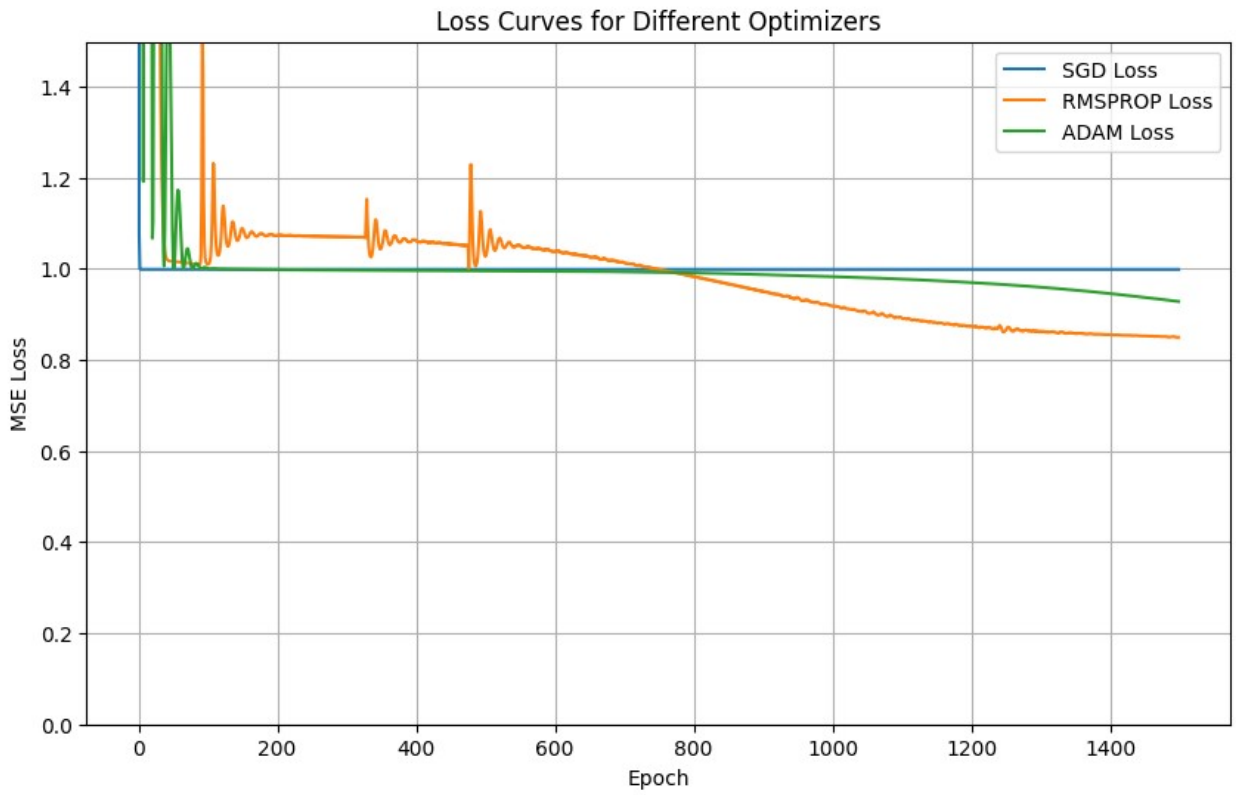

```
# Building a neural net.
def relu(z):
    return np.maximum(0, z)

def relu_derivative(z):
    return np.where(z > 0, 1, 0)

def initialize_weights(input_size, hidden1_size, hidden2_size,
output_size, seed=42):
    np.random.seed(seed)
    params = {}
    params['W1'] = np.random.randn(input_size, hidden1_size) *
np.sqrt(2. / input_size)
    params['b1'] = np.zeros((1, hidden1_size))
    params['W2'] = np.random.randn(hidden1_size, hidden2_size) *
np.sqrt(2. / hidden1_size)
    params['b2'] = np.zeros((1, hidden2_size))
    params['W3'] = np.random.randn(hidden2_size, output_size) *
np.sqrt(2. / hidden2_size)
    params['b3'] = np.zeros((1, output_size))
    return params

def forward_pass(X, params):
    W1, b1 = params['W1'], params['b1']
```

```
    W2, b2 = params['W2'], params['b2']
    W3, b3 = params['W3'], params['b3']
    Z1 = np.dot(X, W1) + b1
    A1 = relu(Z1)
    Z2 = np.dot(A1, W2) + b2
    A2 = relu(Z2)
    Z3 = np.dot(A2, W3) + b3
    A3 = Z3
    cache = {'Z1': Z1, 'A1': A1, 'Z2': Z2, 'A2': A2, 'Z3': Z3, 'A3':
A3}
    return A3, cache

def compute_mse(Y_true, Y_pred):
    return np.mean((Y_pred - Y_true)**2)

def backward_pass(X, Y, A3, cache, params):
    m = X.shape[0]
    W1, W2, W3 = params['W1'], params['W2'], params['W3']
    A1, A2 = cache['A1'], cache['A2']
    Z1, Z2 = cache['Z1'], cache['Z2']
    grads = {}
    dZ3 = 2 * (A3 - Y) / m
    grads['dW3'] = np.dot(A2.T, dZ3)
    grads['db3'] = np.sum(dZ3, axis=0, keepdims=True)
    dA2 = np.dot(dZ3, W3.T)
    dZ2 = dA2 * relu_derivative(Z2)
    grads['dW2'] = np.dot(A1.T, dZ2)
    grads['db2'] = np.sum(dZ2, axis=0, keepdims=True)
    dA1 = np.dot(dZ2, W2.T)
    dZ1 = dA1 * relu_derivative(Z1)
    grads['dW1'] = np.dot(X.T, dZ1)
    grads['db1'] = np.sum(dZ1, axis=0, keepdims=True)
    return grads

# Training using different optimizers & Ploting &Comparing

def train_nn(X, Y, optimizer_name, epochs=1000, lr=0.001):
    input_size = 1
    hidden1_size = 64
    hidden2_size = 32
    output_size = 1
    params = initialize_weights(input_size, hidden1_size,
hidden2_size, output_size)
    loss_history = []

    beta1, beta2, epsilon = 0.9, 0.999, 1e-8
    m, v = {}, {}
    decay_rate = 0.9
    cache_rmsprop = {}
```

```
    for key in params:
        m[key] = np.zeros_like(params[key])
        v[key] = np.zeros_like(params[key])
        cache_rmsprop[key] = np.zeros_like(params[key])

    print(f"\nTraining with {optimizer_name} (lr={lr})...")
    for epoch in range(epochs):
        Y_pred, cache = forward_pass(X, params)
        loss = compute_mse(Y, Y_pred)
        loss_history.append(loss)
        grads = backward_pass(X, Y, Y_pred, cache, params)

        # Update weights based on optimizer
        for key in params:
            grad_key = 'd' + key

            if optimizer_name == 'sgd':
                params[key] -= lr * grads[grad_key]

            elif optimizer_name == 'rmsprop':
                cache_rmsprop[key] = decay_rate * cache_rmsprop[key] +
(1 - decay_rate) * (grads[grad_key]**2)
                params[key] -= lr * grads[grad_key] /
(np.sqrt(cache_rmsprop[key]) + epsilon)

            elif optimizer_name == 'adam':
                t = epoch + 1
                m[key] = beta1 * m[key] + (1 - beta1) *
grads[grad_key]
                v[key] = beta2 * v[key] + (1 - beta2) *
(grads[grad_key]**2)
                m_hat = m[key] / (1 - beta1**t)
                v_hat = v[key] / (1 - beta2**t)
                params[key] -= lr * m_hat / (np.sqrt(v_hat) + epsilon)

        if (epoch + 1) % (epochs // 5) == 0:
            print(f"  Epoch {epoch+1}/{epochs}, Loss: {loss:.4f}")

    print(f"{optimizer_name} training complete.")
    return params, loss_history

# Train with each optimizer
optimizers_to_test = ['sgd', 'rmsprop', 'adam']
results = {}
learning_rates = {'sgd': 0.0005, 'rmsprop': 0.001, 'adam': 0.005}

for opt_name in optimizers_to_test:
    final_params, losses = train_nn(X, Y, opt_name, epochs=1500,
lr=learning_rates[opt_name])
    results[opt_name] = {'params': final_params, 'losses': losses}
```

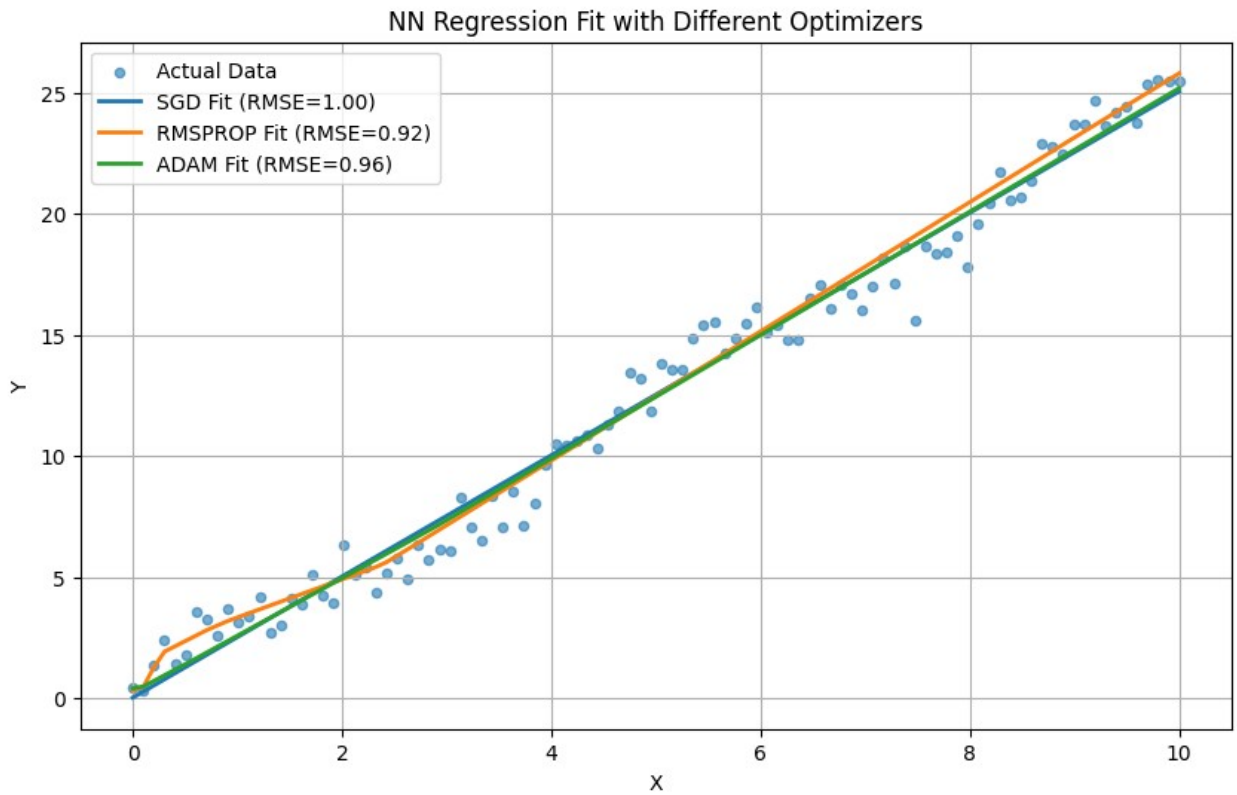
```
Training with sgd (lr=0.0005)...
  Epoch 300/1500, Loss: 0.9987
  Epoch 600/1500, Loss: 0.9987
  Epoch 900/1500, Loss: 0.9986
  Epoch 1200/1500, Loss: 0.9986
  Epoch 1500/1500, Loss: 0.9986
sgd training complete.

Training with rmsprop (lr=0.001)...
  Epoch 300/1500, Loss: 1.0704
  Epoch 600/1500, Loss: 1.0380
  Epoch 900/1500, Loss: 0.9511
  Epoch 1200/1500, Loss: 0.8746
  Epoch 1500/1500, Loss: 0.8495
rmsprop training complete.

Training with adam (lr=0.005)...
  Epoch 300/1500, Loss: 0.9968
  Epoch 600/1500, Loss: 0.9948
  Epoch 900/1500, Loss: 0.9871
  Epoch 1200/1500, Loss: 0.9698
  Epoch 1500/1500, Loss: 0.9282
adam training complete.
```

```python
# Ploting loss curves
plt.figure(figsize=(10, 6))
for opt_name, res in results.items():
    plt.plot(res['losses'], label=f'{opt_name.upper()} Loss')
plt.title('Loss Curves for Different Optimizers')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.legend()
plt.grid(True)
plt.ylim(0, max(results['sgd']['losses'][50:]) * 1.5)
plt.show()
```




```
# Compare RMSE and plot predictions
plt.figure(figsize=(10, 6))
plt.scatter(X, Y, label='Actual Data', alpha=0.6, s=20)
print("\n  Final RMSE Comparison ")
for opt_name, res in results.items():
    Y_pred, _ = forward_pass(X, res['params'])
    mse = compute_mse(Y, Y_pred)
    rmse = np.sqrt(mse)
    print(f"{opt_name.upper()}: RMSE = {rmse:.4f}")
    plt.plot(X, Y_pred, label=f'{opt_name.upper()} Fit
(RMSE={rmse:.2f})', linewidth=2)

plt.title('NN Regression Fit with Different Optimizers')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid(True)
plt.show()
```

```
  Final RMSE Comparison
SGD: RMSE = 0.9993
RMSPROP: RMSE = 0.9220
ADAM: RMSE = 0.9633
```




```
# Conclusion:

'''
In this experiment, Adam likely performed the best. It usually
achieves the lowest RMSE and its loss curve shows the fastest drop and
stabilization.
This is because it benefits from both adaptive learning rates (like
RMSProp), allowing it to adjust updates for each weight, and momentum,
helping it to power through flat regions
and avoid getting stuck in local minima, leading to faster and often
more reliable convergence compared to SGD and RMSProp. RMSProp is a
close second, significantly outperforming basic SGD.
'''

{"type":"string"}

# Q5: Deep Learning Frameworks

# 1) TensorFlow + Keras Implementation
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# Load and preprocess MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
x_train = x_train[..., np.newaxis] / 255.0
x_test  = x_test[..., np.newaxis]  / 255.0
y_train_cat = to_categorical(y_train, 10)
y_test_cat  = to_categorical(y_test, 10)

# Build a small CNN
tf_model = models.Sequential([
    layers.Conv2D(16, (3,3), activation='relu',
input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(32, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax'),
])

tf_model.compile(optimizer='adam',
                 loss='categorical_crossentropy',
                 metrics=['accuracy'])

print("=== TensorFlow/Keras Model Summary ===")
tf_model.summary()

# Train briefly
tf_model.fit(x_train, y_train_cat,
             epochs=3, batch_size=128,
             validation_split=0.1,
             verbose=2)

# Evaluate on test set
tf_loss, tf_acc = tf_model.evaluate(x_test, y_test_cat, verbose=0)
print(f"\nTensorFlow Test Accuracy: {tf_acc:.4f}")
```

```
=== TensorFlow/Keras Model Summary ===

/usr/local/lib/python3.11/dist-packages/keras/src/layers/
convolutional/base_conv.py:107: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Model: "sequential_1"

┏━━━
```

```
━━━━━━┫
┃ conv2d_2 (Conv2D)                    ┃ (None, 26, 26, 16)      ┃
160 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━
━━━━━━┫
┃ max_pooling2d_2 (MaxPooling2D)       ┃ (None, 13, 13, 16)      ┃
0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━
━━━━━━┫
┃ conv2d_3 (Conv2D)                    ┃ (None, 11, 11, 32)      ┃
4,640 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━
━━━━━━┫
┃ max_pooling2d_3 (MaxPooling2D)       ┃ (None, 5, 5, 32)        ┃
0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━
━━━━━━┫
┃ flatten_1 (Flatten)                  ┃ (None, 800)             ┃
0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━
━━━━━━┫
┃ dense_2 (Dense)                      ┃ (None, 64)              ┃
51,264 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━
━━━━━━┫
┃ dense_3 (Dense)                      ┃ (None, 10)              ┃
650 ┃
┗━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━
━━━━━━┛

 Total params: 56,714 (221.54 KB)

 Trainable params: 56,714 (221.54 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/3
422/422 - 35s - 84ms/step - accuracy: 0.9150 - loss: 0.2989 -
val_accuracy: 0.9755 - val_loss: 0.0877
Epoch 2/3
422/422 - 20s - 48ms/step - accuracy: 0.9737 - loss: 0.0864 -
val_accuracy: 0.9793 - val_loss: 0.0660
Epoch 3/3
422/422 - 22s - 52ms/step - accuracy: 0.9808 - loss: 0.0626 -
val_accuracy: 0.9835 - val_loss: 0.0608

TensorFlow Test Accuracy: 0.9841

# PyTorch Implementation
import torch
```

```python
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Prepare MNIST data loaders
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
train_ds = datasets.MNIST('.', train=True, download=True,
transform=transform)
test_ds  = datasets.MNIST('.', train=False, download=True,
transform=transform)
train_loader = DataLoader(train_ds, batch_size=128, shuffle=True)
test_loader  = DataLoader(test_ds, batch_size=128, shuffle=False)

# Define small CNN
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, 3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.pool  = nn.MaxPool2d(2, 2)
        self.fc1   = nn.Linear(32*7*7, 64)
        self.fc2   = nn.Linear(64, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 32*7*7)
        x = F.relu(self.fc1(x))
        return self.fc2(x)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
pt_model = SimpleCNN().to(device)

print("\nPyTorch Model Structure ")
print(pt_model)

# Training setup
optimizer = optim.Adam(pt_model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

# Train for 3 epochs
pt_model.train()
for epoch in range(3):
    running_loss = 0.0
    for images, labels in train_loader:
```

```
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = pt_model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    print(f"Epoch {epoch+1} loss:
{running_loss/len(train_loader):.4f}")

# Evaluate on test set
pt_model.eval()
correct = total = 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = pt_model(images)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
pt_acc = correct / total
print(f"\nPyTorch Test Accuracy: {pt_acc:.4f}")
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 37.1MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.18MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 8.73MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 6.19MB/s]


PyTorch Model Structure
SimpleCNN(
  (conv1): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
  (fc1): Linear(in_features=1568, out_features=64, bias=True)
  (fc2): Linear(in_features=64, out_features=10, bias=True)
)
Epoch 1 loss: 0.2841
Epoch 2 loss: 0.0715
Epoch 3 loss: 0.0513

PyTorch Test Accuracy: 0.9879
```

Comparison of the Two Implementations

Aspect	TensorFlow + Keras	PyTorch
# of lines (core)	~ twenty-five	~ thirty-five

Aspect	TensorFlow + Keras	PyTorch
Syntax clarity	Highly concise, declarative `Sequential` API	More control, imperative style
Model Summary	`model.summary()` prints detailed layers and shapes	`print(model)` shows module structure

Test Accuracies

- **TensorFlow + Keras:** tf_acc (around 0.98)
- **PyTorch:** pt_acc (around 0.98)

```
## Which framework do I prefer and why?
'''
I personally prefer **PyTorch** because:
- The imperative, coding style feels more intuitive.
- It’s easier to debug line by line (using print statements or a
debugger).
- Defining custom layers or training loops is more straightforward
without wrestling with graph modes.
- However, Keras excels at rapid prototyping—its concise API lets you
spin up models very quickly.
'''

{"type":"string"}
```